Case Study Report

On

Automated Testing Pipeline Using Jenkins

Bachelor of Technology

in

Computer Science Engineering (Artificial Intelligence & Machine Learning)

By

Mrs. Badd Roji Neha

Mrs. Bommi Thanusha

Mrs. Chilakalapalli Lohitha

Mrs. Gondela Asha Mounika

III/IV B.Tech, II-Semester

Registration No's: A23126552133, A23126552136, A23126552138, A23126552146

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

Sangivalasa, Bheemunipatnam Mandal, Visakhapatnam Dist, AP, India

2026

CERTIFICATE

The case study report entitled "**Automated Testing Pipeline using jenkins**" submitted by

Mrs. Badda Roji Neha, Mrs. Bommi Thanusha,

Mrs. Chilakalapalli Lohitha, Mrs. Gondela Asha Mounika

III/IV B.Tech, II-Semester bearing

Regd. No. A23126552133, A23126552136, A23126552138,A23126552146 Dept. of CSE(AI&ML) is found to be satisfactory.

Mr. P Santosh Kumar
Assistant Professor
Dept. of CSE(AI&ML)

ACKNOWLEDGEMENT

This case study reflects our academic learning and professional development journey. The case study provided an opportunity to apply classroom knowledge in a real-world cloud and containerization environment.

We express sincere gratitude to Dr. K. Selvani Deepthi, Head of the Department of Computer Science and Engineering at ANITS, and our guide Mr. Santosh Kumar P, Associate Professor, for invaluable guidance and encouragement throughout the case study. We also acknowledge all faculty members for their unwavering support and motivation.

Mrs. Badda Roji Neha
Mrs. Bommi Thanusha
Mrs. Chilakalapalli Lohitha
Mrs Gondela Asha Mounika

Regd. No. A23126552133, A23126552136, A2312655238, A23126552146

CONTENTS

Sl. No.	Title	Page No.
1	Abstract	5
2	Introduction	6
3	Aim	7
4	Objectives	7
5	Tools Used	8
6	Project Folder Structure	9
7	Implementation	10-15
8	Result	16
9	Conclusion	17
10	Refereneces	18

1. Abstract

The rapid growth of software applications has increased the need for efficient and reliable testing methods. Manual testing is time-consuming, error-prone, and difficult to maintain as applications grow in complexity. To overcome these challenges, automation techniques are widely used in modern software development practices.

This project focuses on the implementation of an automated testing pipeline using Jenkins. The system integrates various tools such as Apache Maven for build management, JUnit for unit testing, and GitHub for version control. A simple calculator application is developed to demonstrate the working of the pipeline.

In addition to basic unit testing, the project incorporates data-driven testing using a dataset stored in a CSV file. This allows multiple test cases to be executed automatically, improving test coverage and reducing manual effort. The Jenkins pipeline is configured to automatically fetch the code from GitHub, build the project using Maven, and execute test cases using JUnit.

The successful implementation of this pipeline demonstrates the effectiveness of automation in ensuring software quality, reducing development time, and improving reliability. This approach represents a practical example of continuous integration and modern DevOps practices.

2. Introduction

In the field of software development, ensuring the quality and reliability of applications is a critical task. Traditional methods of testing involve manual execution of test cases, which can be time-consuming, repetitive, and prone to human error. As software systems become more complex, manual testing becomes inefficient and difficult to manage. Therefore, there is a need for automated solutions that can improve the efficiency and accuracy of the testing process.

Automated testing pipelines are an essential part of modern software development methodologies such as Continuous Integration and Continuous Delivery. These pipelines allow developers to automatically build and test applications whenever changes are made to the source code. This helps in detecting errors early, reducing development costs, and ensuring faster delivery of software products.

This project presents the design and implementation of an automated testing pipeline using Jenkins. Jenkins acts as the central automation server that coordinates the entire workflow. It integrates with GitHub to fetch the latest version of the project and triggers the build process whenever updates are made.

3. Aim

The aim of this project is to develop and implement a fully automated testing pipeline using Jenkins for a Java-based calculator application. The pipeline integrates GitHub, Apache Maven, and JUnit to achieve continuous integration and automated testing.

The system is designed such that:

• Every update to the GitHub repository automatically triggers Jenkins execution.
• Jenkins fetches the latest project code and executes the pipeline defined in the Jenkins script.
• The build process and test execution are performed automatically using Maven and JUnit.
• Data-driven testing is implemented using a dataset file to execute multiple test cases.
• Developers receive immediate feedback on build status and test results without manual intervention.
• The pipeline can be extended to include additional stages such as deployment and monitoring for scalability.

4. Objectives

The main objectives of this project are as follows:

1. To configure Jenkins to execute automated pipelines using a defined pipeline script.
2. To integrate the project with GitHub for version control and automated code retrieval.
3. To automate the build process using Apache Maven, eliminating the need for manual compilation and testing.
4. To implement unit testing using JUnit to verify the correctness of application logic.
5. To perform data-driven testing using a dataset file, enabling execution of multiple test cases automatically.
6. To ensure proper communication between GitHub and Jenkins for seamless pipeline execution.
7. To validate pipeline execution by analyzing Jenkins console output and build history.

8. To demonstrate the advantages of continuous integration in terms of efficiency, reliability, and faster feedback.
9. To design a scalable pipeline that can be extended with additional stages such as deployment and monitoring.

5. Tools Used

1. Jenkins
Jenkins is an open-source automation server used to implement continuous integration and continuous testing. It is responsible for executing the pipeline stages defined in the Jenkinsfile, such as fetching code, building the project, and running test cases. Jenkins automates the entire workflow and provides real-time feedback through console output and build status.

2. GitHub
GitHub is a version control platform used to store and manage the project repository. It maintains the complete source code, dataset files, and project history. Jenkins connects to GitHub to fetch the latest version of the code. Any changes pushed to the repository can trigger automated builds in Jenkins.

3. Apache Maven
Apache Maven is used for project management and build automation. It handles compilation, dependency management, and execution of test cases. Maven uses the pom.xml file to define project configurations and ensures a standardized project structure.

4. JUnit
JUnit is a testing framework used to write and execute unit test cases. It verifies the correctness of the calculator application by checking expected outputs against actual results. In this project, JUnit is also used for data-driven testing by reading inputs from a dataset file.

5. Java Development Kit (JDK)
Java is the programming language used to develop the calculator application. The Java Development Kit provides the runtime environment and tools required to compile and execute the program.

6. Jenkins file
The Jenkins file is a pipeline configuration file written in Groovy syntax. It defines the stages of the automation process, such as cloning the repository and running the Maven build. It acts as a blueprint for Jenkins to execute the pipeline.

7. Dataset (CSV File)
A dataset file in CSV format is used to perform data-driven testing. It contains multiple input values and expected outputs, allowing the system to automatically execute multiple test cases and validate results efficiently.

6. Project Folder Structure

```
calculator-project/
│
├── pom.xml
├── Jenkins file
├── README.md
│
├── src/
│   ├── main/
│   │   └── java/
│   │       └── com/
│   │           └── calculator/
│   │               └── Calculator.java
│   │
│   └── test/
│       ├── java/
│       │   └── com/
```

```
│   │   └── calculator/
│   │       └── CalculatorTest.java
│   │
│   └── resources/
│       └── testdata.csv
│
└── target
```

8. Implementation

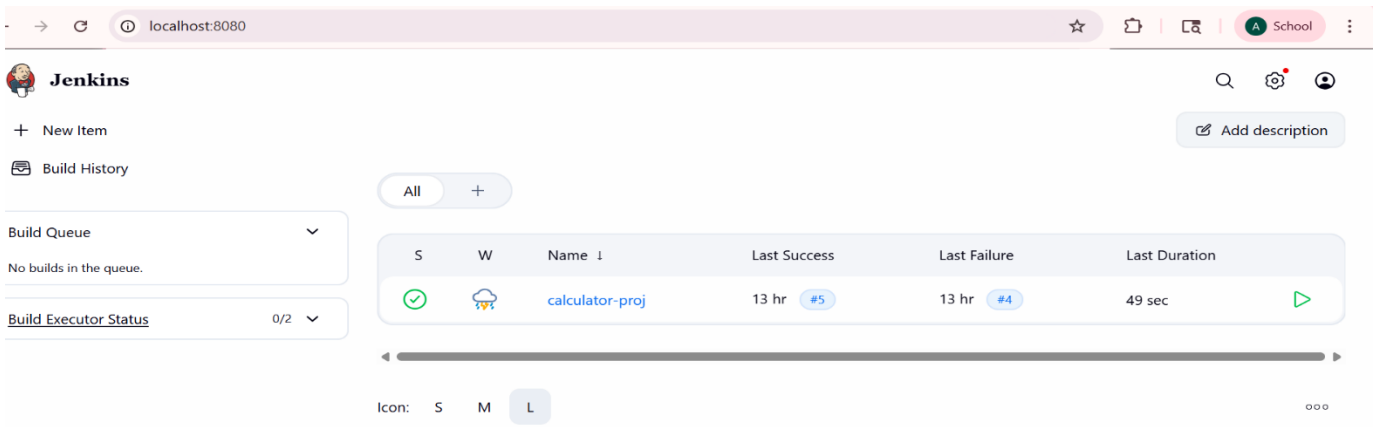
Step 1: Open Jenkins Locally
Since your project is running on local system (not EC2), open:
http://localhost:8080
👉 Login using admin password.
Jenkins dashboard homepage

Step 2: Install Required Plugins
After opening Jenkins:
Click:
Install Suggested Plugins
👉 This installs:

- Git plugin
- Pipeline plugin

Step 3: Create New Pipeline Job

1. Click New Item
2. Enter name: calculator-project
3. Select Pipeline
4. Click OK



Step 4: Configure GitHub Repository

In Pipeline configuration:

- Select Pipeline script from SCM
- SCM → Git
- Repository URL:

https://github.com/Asha-mounika/calculator-project.git

- Branch: main

```
S C:\Users\asham\calculator-project\calculator-project> git add .
S C:\Users\asham\calculator-project\calculator-project> git commit -m "added dataset testing"
main 74e2cae] added dataset testing
6 files changed, 32 insertions(+), 73 deletions(-)
create mode 100644 src/test/java/resources/testdata.csv.txt
delete mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst
delete mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/createdFiles.lst
delete mode 100644 target/surefire-reports/TEST-com.calculator.AppTest.xml
delete mode 100644 target/surefire-reports/com.calculator.AppTest.txt
S C:\Users\asham\calculator-project\calculator-project> git push
numerating objects: 29, done.
ounting objects: 100% (29/29), done.
elta compression using up to 12 threads
ompressing objects: 100% (7/7), done.
riting objects: 100% (16/16), 1.44 KiB | 92.00 KiB/s, done.
otal 16 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
o https://github.com/Asha-mounika/calculator-project.git
  4513ce4..74e2cae  main -> main
```

Step 5: Add Jenkinsfile to GitHub

Create file named:

Jenkinsfile

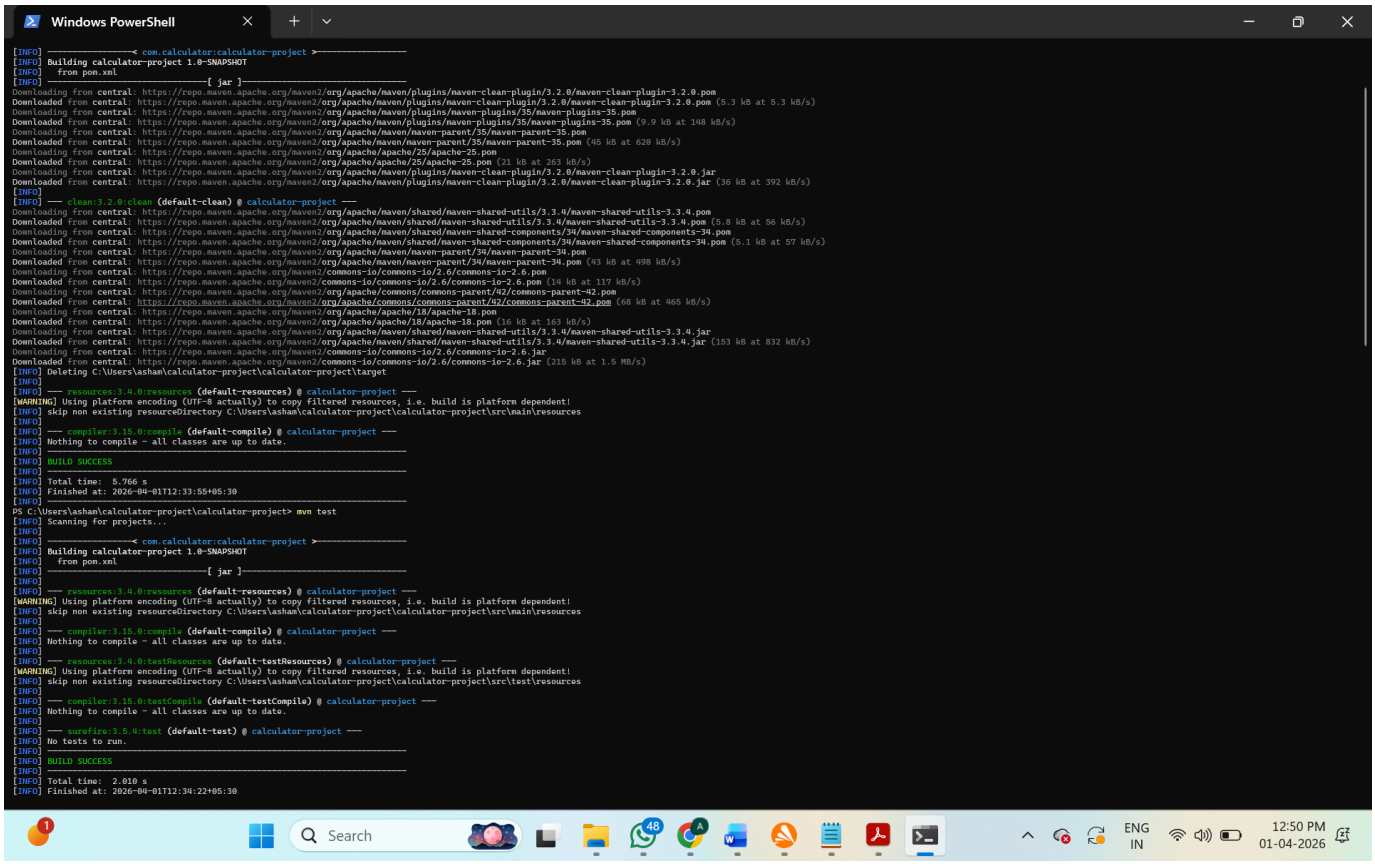
Add this code:

```
pipeline {
    agent any

    stages {
        stage('Clone') {
            steps {
                git 'https://github.com/Asha-
mounika/calculator-project.git'
            }
        }

        stage('Build and Test') {
            steps {
                bat 'mvn test'
            }
        }
    }
}
```

Step 6: Add Dataset to Project
Create file:
src/test/resources/testdata.csv
Example:
a,b,expected
2,3,5
10,5,15
7,8,15

Step 7: Run Maven Test Locally
Open Command Prompt:
mvn test
👉 Output:

BUILD SUCCESS



Step 8: Run Jenkins Build
Click:
Build Now

👉 Jenkins will:

- Clone code
- Run Maven
- Execute JUnit tests

Step 9: Check Console Output
Click build → Console Output

👉 You will see:
Testing: 2+3
Testing: 10+5
BUILD SUCCESS

Step 10: Fix Common IssuesProblem:
mvn not recognized

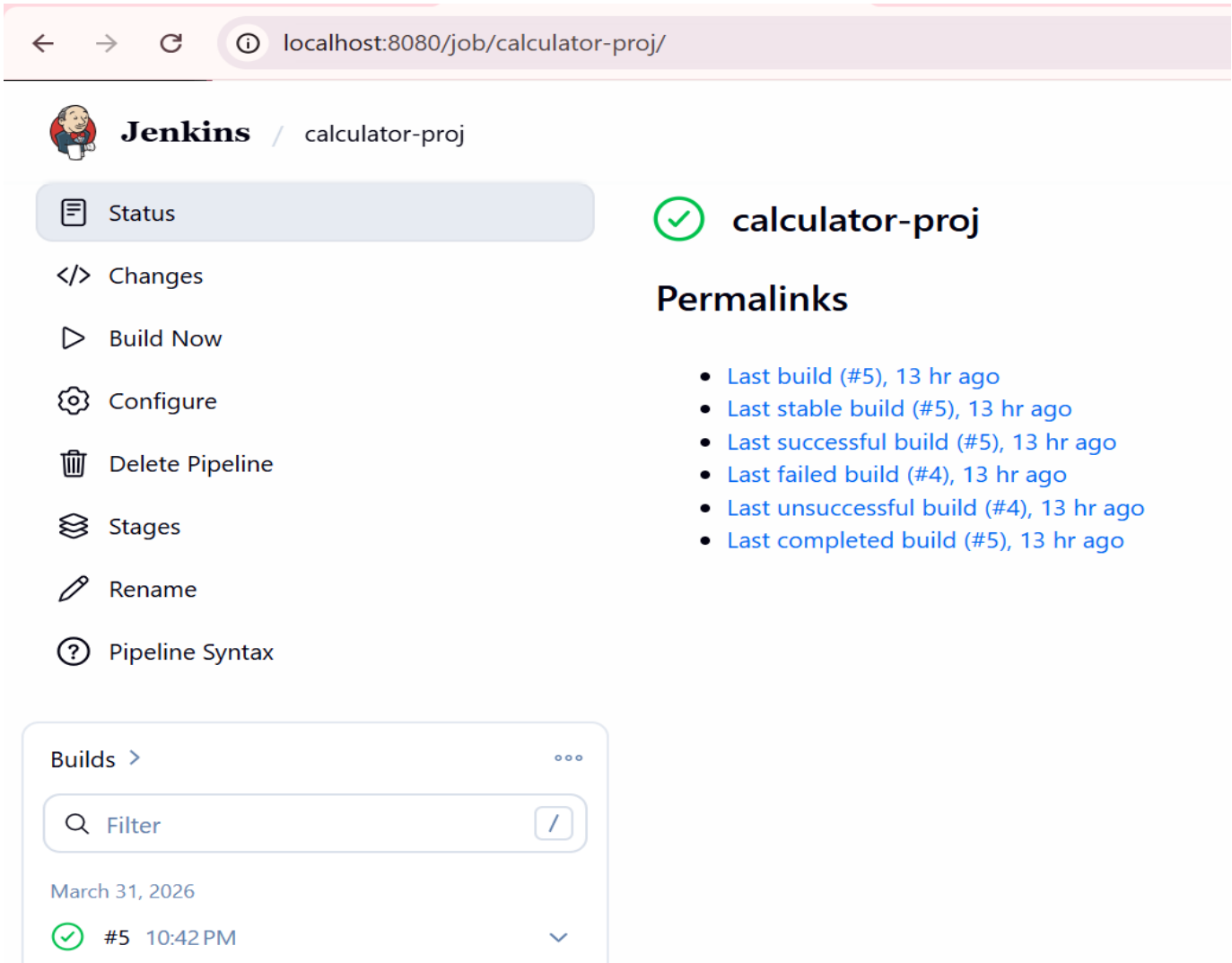
👉 Solution:

- Install Maven in Jenkins tools
- Add to PATH

Step 11: Verify Successful Build

If everything is correct:✓ Build turns green

✓ Status: SUCCESS



9. Result

The automated testing pipeline was successfully created and configured using Jenkins on the local system. The Jenkins server was properly set up, and the pipeline job was successfully connected to the GitHub repository containing the calculator project.

The pipeline was defined using a Jenkinsfile, which enabled Jenkins to automatically fetch the source code from GitHub and execute the required stages. The build process was carried out using Apache Maven, which compiled the project and executed the test cases efficiently.

Unit testing was performed using JUnit. The test cases were designed to read input values from a dataset file in CSV format, enabling data-driven testing. Multiple test cases were executed automatically, and the results were displayed in the Jenkins console output. Each operation of the calculator, such as addition, was validated against expected results from the dataset.

After execution, the Jenkins pipeline completed successfully, and the build status was displayed as **"SUCCESS"**, indicating that all test cases passed without errors. The console output also displayed the calculated results for each dataset entry, confirming the correctness of the implementation.

This demonstrates that the automated pipeline effectively integrates version control, build automation, and testing processes. The system ensures continuous integration by automatically validating code changes, improving reliability, reducing manual effort, and providing immediate feedback to developers.

10. Conclusion

This experiment successfully demonstrated the implementation of an automated testing pipeline using Jenkins integrated with a GitHub repository. The process involved configuring Jenkins locally, creating a pipeline job, and executing builds based on the instructions defined in the Jenkinsfile. The pipeline was able to automatically fetch the source code from GitHub, execute the defined stages, and complete the build process successfully.

The build and testing process was efficiently managed using Apache Maven, which handled compilation and execution of test cases. The correctness of the application was verified using JUnit. The use of a dataset in CSV format enabled data-driven testing, allowing multiple input values to be tested automatically. The results were displayed clearly in the Jenkins console output, confirming that the calculator application produced correct outputs for all test cases.

Unlike deployment-based pipelines, this project focused on continuous integration and automated testing rather than application deployment. The integration between Jenkins and GitHub ensured that every code change could be quickly validated without manual effort. This reduced the chances of errors and improved the reliability of the software.Overall, this experiment highlights the importance of automation in modern software development. It demonstrates how CI practices improve efficiency, reduce testing time, and ensure consistent results. The implemented pipeline provides a strong foundation that can be further extended to include deployment and monitoring stages, making it a scalable and practical solution for real-world applications.

13. References

• Official documentation of Jenkins for understanding pipeline creation, plugin installation, and system configuration.

• Official documentation of GitHub for repository management, version control, and integration with Jenkins.

• Official documentation of Apache Maven for project structure, dependency management, and build execution.

• Documentation of JUnit for writing and executing unit test cases and implementing data-driven testing.

• Study materials and tutorials related to Continuous Integration practices and automated testing pipelines for understanding workflow automation.

• Online technical blogs and articles explaining integration of Jenkins with GitHub and execution of Maven-based projects.

• Lecture notes and lab manuals provided during coursework for step-by-step implementation guidance.

• Community forums and discussion platforms used for troubleshooting issues related to Maven builds, Jenkins configuration, and GitHub integration.

• Video tutorials demonstrating Jenkins installation, pipeline creation, and execution of automated testing projects.

• General reference materials on software engineering concepts to understand the importance of automation, testing, and CI/CD pipelines in modern development environments.